

LawOnCall

Comprehensive End-to-End QA Test Suite Documentation.

A system-wide master verification workbook designed for QA engineers, developers, project managers, and release stakeholders. Validated under the Digital Personal Data Protection Act, 2023.

1. Project Overview

Project Name: LawOnCall (operated under consumer domain lawoncall.in)

Purpose: LawOnCall is an enterprise-level, high-availability legal tech platform that serves as a technology intermediary under Section 79 of the Information Technology Act, 2000. It connects consumers seeking prompt legal advice with independent, bar-council-registered advocates. The platform handles end-to-end matching, call masking, billing settlements, document generation, and admin monitoring.

Scope: This documentation covers the verification of the React PWA frontend (client & advocate dashboards), the Fastify-based microservices architecture (Gateway, Auth, Lead, Payment, Notification, Profile), database schemas (PostgreSQL RDS), and external integration APIs (Exotel telephony, Razorpay/PhonePe payment processing, Firebase OTP authentication, and FCM push notifications).

Objectives: Achieve 100% automated matches with sub-minute latency; ensure privacy compliance with the Indian Digital Personal Data Protection Act, 2023 (DPDP Act); establish 99.9% billing accuracy; and monitor advocates' Service Level Agreement (SLA) indicators globally.

Technology Stack:

- **Frontend:** React, TypeScript, Vite, Tailwind CSS, Service Workers (PWA), LocalForage.
- **Backend:** Node.js, Fastify microservice framework, Prisma ORM, Redis session cache.
- **Database:** PostgreSQL on AWS RDS with automatic write-ahead logs and hourly backups.
- **Infrastructure:** AWS S3 (hosting static assets), CloudFront CDN (global routing), EC2 (microservices application server), Nginx (reverse proxy & SSL termination via Certbot), AWS IAM (role constraints).
- **Integrations:** Exotel APIs (call masking, masked call records, callback webhooks), Firebase (auth verification, OTP, push alerts), PhonePe & Razorpay (transaction processing).

System Architecture Diagram Description:

The system is divided into three isolated zones: Zone A (Public Edge - CloudFront S3 and Nginx Proxy), Zone B (Private Application Layer - Gateway and isolated microservices running inside Docker Containers), and Zone C (Storage Layer - RDS PostgreSQL isolated inside private subnets). Communications between Zone B services use internal DNS / docker network aliases, preventing direct internet snooping.

2. Application Flow

The following description defines the end-to-end process flow from user initiation through system processing to database logging and notification delivery:

2.1 User Registration & Login Flow:

1. User enters mobile number in React PWA Client.
2. Frontend sends request to Gateway service: POST /api/auth/send-otp.
3. Gateway checks rate limits (IP and phone keys in Redis), then proxies payload to Auth Service.
4. Auth Service triggers Firebase Phone Auth REST API to dispatch SMS OTP.
5. User enters OTP code. PWA calls Gateway: POST /api/auth/verify-otp.
6. Auth Service validates OTP via Firebase verification endpoint. If success, Auth Service queries the PostgreSQL database via Prisma to check if user exists. If yes, fetches details; if no, creates user record in User table.
7. Auth Service generates a signed JWT access token (valid for 15 minutes) and a refresh token (valid for 7 days), returning them to client. User state changes to 'Authenticated'.

2.2 ASAP Calling and Matching Flow:

1. Authenticated User navigates to dashboard, selects case category, and clicks 'Call Now'.
2. Client triggers: POST /api/leads/book-asap with JWT authorization header.
3. Gateway validates JWT signature using public secret key, extracts user payload, and proxies request to Lead Service.
4. Lead Service creates a lead record in Lead table with status set to PENDING. It queries Advocate table for active, online, and unassigned advocates matching the case category.
5. Lead Service matches the user with the best-ranking advocate and locks the advocate state in database. It updates lead status to ASSIGNED.
6. Lead Service triggers the Exotel Masked Call API: POST /api/exotel/call, passing the advocate's phone number, user's phone number, and a callback URL.
7. Exotel server rings both parties, masking real phone numbers. Neither client nor advocate can see each other's contact number.
8. Exotel sends status callback to Payment Service to log transaction time and call metadata. Payment Service deducts minutes from user's active wallet package inside Wallet table.

2.3 Admin Case Management & SLA Monitors:

1. Admin logs into the admin portal. Gateway verifies the ADMIN role from the JWT token.
2. Admin accesses leads, checks SLA tags (e.g. Not Attended for calls not connected in 60s, or Manual Handled for manual lawyer override).
3. Admin can post updates in Comment table and change lead status to Case Closed, Case Cancelled, or Case Aborted. System triggers webhooks to update databases.

3. Module-wise Breakdown

M1: User Authentication & Registration Module

Description: Manages user verification, registration, JWT generation, and token refresh.

Business Logic: Validates Indian mobile format (+91 or 10 digits). Queries database via Prisma. Issues access/refresh JWT tokens. Encrypts session payloads.

Input Fields: Mobile Number (string), OTP Code (6-digit numeric), Refresh Token (JWT string).

Validations: Mobile: Must match regular expression `^[6-9][0-9]{9}$`. OTP: Must be exactly 6 digits, numeric only.

Dependencies: Firebase Auth API, PostgreSQL Database, Redis token blacklist.

Expected Outputs: 200 OK with token payload on success; 401 Unauthorized for bad OTP; 400 Bad Request for incorrect input format.

Error Scenarios: Database connection drop (fallback to 500 error screen), SMS gateway API timeout (trigger retry client button).

M2: Booking & Lawyer Matching Module

Description: Matches customers with available advocates on-demand or on schedule.

Business Logic: Fetches online advocates with corresponding category tags. Sorts by ranking and SLA metrics. Assigns lead to lawyer and triggers Exotel calling bridge.

Input Fields: Case Category (enum), Location/City (string), Preferred Slot (optional datetime), User Wallet ID (UUID).

Validations: Category: Must match predefined system enums. Wallet ID: Must represent an active wallet with at least 15 minutes balance.

Dependencies: Profile Service (advocate status), Payment Service (wallet balance).

Expected Outputs: Match success, Lead status set to ASSIGNED, connection details returned.

Error Scenarios: No advocates online (return 404 No Advocates Available and push user to waitlist), Database locked on advocate conflict (retry matching algorithm up to 3 times).

M3: Masked Telephony & Exotel Interface Module

Description: Initiates, bridges, and logs phone calls without exposing real numbers.

Business Logic: Calls Exotel virtual connect API. Tracks call start, duration, end, and call status. Triggers webhook on call termination.

Input Fields: User Phone (masked), Advocate Phone (masked), Max Call Duration (integer).

Validations: Phones: Must be in valid E.164 formats. Duration: Must match active wallet minutes remaining.

Dependencies: Exotel Telephony Gateway, Lead matching system.

Expected Outputs: Exotel call connection bridged successfully. Masked caller ID displayed.

Error Scenarios: Exotel API timeout (abort call, refund wallet, mark lead as SYSTEM_FAIL), Webhook payload signature validation fail (discard webhook request, log threat alert).

M4: Payment & Wallet Settlement Module

Description: Processes customer credit/debit card, UPI, and net banking transactions, manages talk-time wallets.

Business Logic: Authenticates with Razorpay/PhonePe. Manages webhook notifications. Updates user wallet balance upon successful credit.

Input Fields: Payment Amount (numeric), Package Type (enum), Transaction ID (string), Razorpay Signature (string).

Validations: Amount: Positive numeric. Signature: Must match calculated SHA-256 HMAC of transaction body.

Dependencies: Razorpay API, PhonePe API, Wallet Database table.

Expected Outputs: Payment status verified, wallet minutes credited immediately, receipt sent.

Error Scenarios: Payment gateway callback timeout (re-run cron check within 1 hour), duplicate webhook callback (ignore second call based on Transaction ID primary key constraint).

4. Complete Test Strategy

Our testing strategy ensures that every component is validated before release. Below is our testing matrix:

Testing Type	Target Objective	Tools & Frameworks	Methodology & Execution
Functional	Verify signup, login, bookings, calling, payments, admin panels.	Jest, Playwright, Supertest	Automated integration tests on each build. Manual validation of flows.
UI/UX	Ensure PWA layout responsiveness across mobile, tablet, and desktops.	Playwright, Appium, BrowserStack	Cross-browser rendering tests (Chrome, Safari, Firefox, Edge).
API	Validate endpoints, payloads, HTTP codes, rate limiters, token filters.	Postman, Newman, Supertest	Run Postman test collections with bad tokens, missing params.
Database	Check CRUD actions, foreign key constraints, locks, duplicate records.	Prisma, PGUnit, SQL scripts	Verify rollbacks on transaction failure. Confirm indexes are utilized.
Security	OWASP Top 10, SQLi, XSS, JWT hijacking, role violations.	OWASP ZAP, Burp Suite, Snyk	Inject script payloads in form inputs. Bypass routing to admin URLs.
Performance	Validate gateway response latency, call routing load times.	JMeter, k6	Simulate 1000 concurrent user matching requests. Measure DB CPU.
Regression	Verify updates do not break core services, calling or wallet checks.	CI/CD GitHub Actions pipelines	Run entire automated test suite on every pull request to main.

5. Detailed Test Cases

This workbook maps exhaustive system test cases. Standard columns: ID, Module, Scenario, Preconditions, Steps, Expected, Status, Priority, Severity.

ID	Module	Scenario	Steps	Expected Result	Stat	Pri	Sev
TC-01	Auth	Valid OTP signup	1. Enter +919999999999. 2. Send OTP. 3. Verify correct OTP.	User created in database, JWT issued, redirect to dashboard.	PASS	H	C
TC-02	Auth	Signup invalid phone format	1. Enter '12345' in phone field. 2. Click Send OTP.	Client validation blocks submit. Input error banner shows.	PASS	H	M
TC-03	Auth	Verify with expired OTP code	1. Request OTP. 2. Wait 11 mins. 3. Submit OTP.	System rejects code, displays 'OTP Expired' message.	PASS	M	H
TC-04	Matching	ASAP call, advocate available	1. User wallet has 30 mins. 2. Select Family Law. 3. Click Call Now.	Online Family Law lawyer located, matched, calls triggered.	PASS	H	C
TC-05	Matching	ASAP call, no advocates online	1. Select Cyber Law. 2. Click Call Now. 3. No active lawyer available.	Matches fail gracefully, redirects client to waitlist page.	PASS	H	H
TC-06	Matching	ASAP call, zero wallet balance	1. Wallet balance is 0 mins. 2. Select Family Law. 3. Click Call Now.	System blocks request, pops modal: 'Recharge wallet to call'.	PASS	H	H
TC-07	Calling	Exotel call masking trigger	1. Match advocate. 2. Initiate call API. 3. Inspect virtual numbers.	Exotel routes call via masked numbers. Real numbers are hidden.	PASS	H	C
TC-08	Calling	Exotel API gateway timeout	1. Block Exotel endpoints. 2. Click Call Now.	System catches error, marks lead as FAIL, refunds wallet minutes.	PASS	M	H
TC-09	Payments	PhonePe webhook success log	1. Submit pay of Rs 500. 2. Gateway sends success webhook.	Webhook hash verified, wallet updated with 30 mins, logs saved.	PASS	H	C
TC-10	Payments	Duplicate webhook callback	1. Send success webhook. 2. Send duplicate webhook.	System ignores second webhook due to transaction primary key constraint.	PASS	M	H
TC-11	Admin	Global Case status update	1. Admin logs in. 2. Selects Lead #102. 3. Set status: Case Closed.	Database updates lead status to 'Case Closed'. Webhook triggered.	PASS	H	H
TC-12	Security	SQL injection in Name input	1. In signup name input type: ' OR 1=1; --. 2. Submit.	Prisma queries sanitize string. Text saved literally. Injection fails.	PASS	H	C
TC-13	Security	XSS script injection attempt	1. In case info field type: <script>alert(1)</script>. 2. Submit.	HTML encoded in DB and escaped upon rendering. Script does not run.	PASS	H	C

ID	Module	Scenario	Steps	Expected Result	Stat	Pri	Sev
TC-14	Security	Bypass URL to Admin panel	1. Log in as client. 2. Force browser URL: /admin/dashboard.	Client routing blocks page. Gateway checks JWT role, returns 403.	PASS	H	C
TC-15	Databas e	Prisma transaction rollback	1. Start match transaction. 2. Force db crash after step 1.	All matched states rollback. Advocate online status restored.	PASS	H	H

6. Form Validation Testing

The system enforces field validations on the PWA frontend and within each Fastify microservice schema. The validation criteria for each input field are detailed below:

Field Name	Validation Test Scenario	Expected System Behavior & Response
Mobile Number	Empty / Null value submit.	Frontend prevents submission. Alert displays: 'Mobile number is required'.
Mobile Number	Invalid formats (letters, less than 10 digits).	Blocked. Display: 'Please enter a valid 10-digit mobile number'.
Mobile Number	Unicode characters, emojis, leading/trailing spaces.	Leading/trailing spaces are auto-trimmed. Emojis and letters are stripped. Gateway returns 400 if passed.
Full Name	SQL Injection attempt: ' OR 1=1; --	Prisma parameterization sanitizes inputs. String is saved literally as literal text name. DB is safe.
Full Name	XSS Injection attempt: <script>alert(1)</script>	String is saved literally. React escapes variable values during DOM injection, preventing script execution.
Full Name	Exceed maximum length (e.g. 150+ chars).	Blocked by client-side max-length limit (100). Microservice schema returns 400 validation error.
OTP Code	Incorrect length (e.g. 4 or 5 digits).	Blocked. Validation error: 'OTP must be exactly 6 digits'.
Case Details	Emojis and unicode characters included.	Allowed. Database supports UTF-8 encoding. Characters are rendered correctly in the admin dashboard.

7. Authentication & Authorization Testing

Security of user credentials and role access is paramount. The following test cases validate JWT security, role access control (RBAC), and session timeouts:

Security Scenario	Test Procedure	Expected Security Response
JWT Verification & Tampering	Modify the signature payload of a valid client access JWT token, then send request to GET /api/leads.	Fastify Gateway gateway intercepts, signature verification fails. Gateway returns 401 Unauthorized and drops request.
Access Token Expiration	Set JWT access token lifetime to 1 second. Wait 2 seconds. Trigger request to profile API.	Gateway rejects request with 401 Token Expired. Client interceptor catches, fires refresh token, gets new JWT.
Role-Based Access Control (RBAC)	Login as standard client user. Retrieve JWT token. Attempt request to admin endpoint: GET /api/admin/leads.	Gateway extracts user role from token (sees role is USER instead of ADMIN), blocks access, returns 403 Forbidden.
Direct URL Access Bypass	Clear browser storage (localStorage, cookies). Open a new tab, navigate directly to URL path: /admin/dashboard.	React Router interceptor checks local storage for active session, sees empty token, blocks page render, redirects to /login.
Concurrent Session Management	Log in on Device A. Log in on Device B with the same user credentials. Trigger request from Device A.	The login on Device B triggers database session token update. Device A token is invalidated in Redis session store, returns 401.

8. API Test Cases

All API endpoints must return precise JSON payloads, enforce authentication constraints, and support rate limiting. Endpoints and their expected validation structures are detailed below:

Endpoint & Method	Headers & Request Body	Expected Response Code & JSON Verification
POST /api/auth/send-otp	Headers: Content-Type: application/json Body: { "phone": "+919999999999" }	Code: 200 OK JSON: { "success": true, "message": "OTP sent successfully" }
POST /api/auth/verify-otp	Headers: Content-Type: application/json Body: { "phone": "+919999999999", "code": "123456" }	Code: 200 OK JSON: { "token": "eyJhbGciOi...", "refreshToken": "..." }
POST /api/leads/book-asap	Headers: Authorization: Bearer <token> Body: { "category": "FAMILY_LAW" }	Code: 201 Created JSON: { "leadId": "uuid-string", "status": "ASSIGNED", "exophone": "+9180..." }
POST /api/payments/webhook	Headers: x-razorpay-signature: <sha256-hmac> Body: Razorpay event transaction object.	Code: 200 OK JSON: { "status": "PROCESSED" }
PUT /api/admin/leads/:id/status	Headers: Authorization: Bearer <admin-token> Body: { "status": "CASE_CLOSED", "comment": "Case completed successfully" }	Code: 200 OK JSON: { "updated": true, "newStatus": "CASE_CLOSED" }

9. Database Test Cases

Database operations must preserve referential integrity, enforce structural constraints, and handle high-concurrency transactions. The table below outlines our database verification cases:

Database Operation	Precondition & Setup	Expected DB Execution & Integrity Outcome
CRUD Lead Creation	User record must exist in User table. Advocate must be online.	Prisma inserts record in Lead table, correctly setting foreign keys (clientId, advocateId) and default status PENDING.
Foreign Key Integrity Constraint	Attempt to insert a lead with a non-existent clientId UUID value.	Prisma throws a P2003 foreign key constraint violation. Database aborts insert, ensuring data integrity.
Concurrent Match Lock	Two users attempt to match with the same online advocate at the exact same millisecond.	Database executes transaction with SERIALIZABLE isolation or row lock. One request succeeds; the other rolls back and retries.
Double Webhook Deduplication	Payment Service receives two webhook receipts containing the same duplicate Transaction ID.	Unique constraint on transaction ID column blocks the second insert. Wallet credits are only computed once.
Prisma Transaction Rollback	Lead matching API succeeds but Exotel call dispatch fails mid-execution.	The entire transaction rolls back. Advocate is returned to ONLINE status, and lead is deleted or set to FAILED.

10. Security Test Cases

To protect our enterprise-level application, we validate safety parameters against OWASP Top 10 vulnerabilities. The test matrix below details these security verification protocols:

OWASP Vulnerability	Attack Vector Scenario	Security Defense & Mitigation Outcome
SQL Injection	Inject UNION SELECT commands into profile search bar query parameters.	All inputs are fully parameterized by Prisma ORM. Commands are treated as string literals, rendering the exploit ineffective.
Cross-Site Scripting (XSS)	Inject Javascript image triggers into Case Summary text areas: <code></code> .	All input scripts are encoded in database. React frontend renders value safely using virtual DOM node escaping, script does not execute.
Broken Authentication	Send brute force OTP requests to POST <code>/api/auth/verify-otp</code> using script loops.	Fastify Gateway rate limiter detects traffic, blocks IP/phone key for 1 hour after 5 failures, returning 429 Too Many Requests.
Broken Object Level Auth	Log in as Client A, try to access case detail of Client B via direct ID URL: GET <code>/api/leads/uuid-b</code> .	Lead Service checks token identity against requested lead record. If mismatch, throws 403 Forbidden.
Sensitive Data Exposure	Snoop network call payloads on public networks during masked consultations.	All communications enforce TLS 1.3 encryption. Masked routing via Exotel keeps real phone numbers hidden on packets.

11. Performance Test Cases

To ensure reliability during spikes, performance testing targets API latencies, database loads, and concurrency thresholds. Verification criteria include:

- **Response Time:** API Gateway response time must be under 200ms for 95% of standard read queries.
- **Load Testing:** 1000 concurrent virtual users simulated using k6 scripts hitting authentication and profile modules for 10 minutes. Threshold: CPU load below 70%, zero packet drops.
- **Stress Testing:** Peak traffic ramped up to 2500 concurrent connections to find breakpoint. Result: System remains stable; auto-scaling EC2 instances launch, DB connection pool scales up safely.
- **DB Query Latency:** PostgreSQL query execution time for advocate availability indexing must be under 50ms. Verified using EXPLAIN ANALYZE.

12. UI/UX Test Cases

To guarantee visual appeal, usability, and responsiveness, the layout has been tested across multiple screens. Key UI test scenarios include:

- **Mobile Breakpoints (375px - 480px):** Verify that dashboard items stacking layout displays properly. Ensure touch target size is at least 44x44px.
- **Tablet Breakpoints (768px - 1024px):** Verify sidebar overlay and card grids render without alignment shifts.
- **Cross-Browser Rendering:** Check layout alignment, font loading, and SVG rendering on Apple Safari, Google Chrome, Mozilla Firefox, and Microsoft Edge.
- **Accessibility:** Verify that color contrast ratio is at least 4.5:1. Confirm keyboard navigation using TAB is functional across all input forms.
- **Error Banners:** Verify that dynamic error popups are styled in red, readable, and disappear after 5 seconds.

13. End-to-End User Journeys

The following user journeys map real-world usage scenarios across multiple system components:

13.1 New Client Onboarding & ASAP Call Journey:

1. User launches the client PWA app, lands on index, and clicks 'Get Started'.
2. Registers via phone number, verifies OTP, and enters name. System generates new profile in DB.
3. User clicks 'Recharge Wallet', selects the 30-minute plan, completes mock Razorpay payment. DB logs transaction, credits minutes.
4. User navigates back to dashboard, selects 'Criminal Law', and clicks 'Call Now'.
5. Match algorithm picks an online, active criminal advocate. Exotel bridges masked call. Advocate answers, call concludes in 10 minutes.
6. Call terminates. Client wallet is deducted by 10 minutes. User is prompted with advocate feedback review.

13.2 Case Recovery & Call Drop Journey:

1. Client and advocate are talking. Mid-call, client loses network connectivity, call drops.
2. Exotel detects call drop, sends webhook payload to Payment Service: status: completed, duration: 240s.
3. Payment Service calculates exact minutes used (4 minutes), returns remaining 26 minutes to user wallet, and sets lead status to UNRESOLVED.
4. User regains network, logs back in, clicks 'Call Support/Reconnect'. System offers to match again or reschedule. User matches again with 0 additional charge.

14. Bug Reporting Template

Our engineering team utilizes a standardized issue reporting template to ensure bugs are resolved rapidly. Below is the enterprise format:

Field	Description & Live Example
Bug ID	BUG-LM-2026-004
Summary	Advocate online status fails to sync with database when network switches.
Steps to Reproduce	1. Log in to advocate dashboard. 2. Toggle status to Online. 3. Disable Wi-Fi to force mobile data switch. 4. Observe status shows online but database marks offline.
Expected Result	Websocket should reconnect, syncing status automatically to 'Online'.
Actual Result	Database remains marked offline; incoming match requests fail.
Severity / Priority	Severity: High / Priority: High
Environment Details	PWA Client v1.2, iPhone 14, iOS 17.2, Safari Browser.

Logs / Payload	WebSocket connection to 'wss://api.lawoncall.in/ws' failed: Connection timed out
----------------	--

15. Test Execution Summary

Below is the execution result summary for the release validation phase of the LawOnCall application:

Category	Total Cases	Passed	Failed	Blocked	Defects Found	Defect Density	Coverage %
Authentication	30	30	0	0	1	0.03	100%
Matching / Call	45	43	2	0	4	0.09	98%
Payments / Wallet	25	25	0	0	0	0.00	100%
Admin Controls	20	19	1	0	1	0.05	95%
Security (OWASP)	15	15	0	0	0	0.00	100%
Total Overall	135	132	3	0	6	0.04	98.5%

16. Test Coverage Matrix

The matrix maps system specifications directly to their respective verification test cases:

Requirement Code	System Feature Description	Mapped Test Case IDs	Coverage Status
REQ-AUTH-001	Authentication via mobile OTP & Firebase validation.	TC-01, TC-02, TC-03	100% Covered
REQ-MATCH-002	Automated lawyer selection and lead state updates.	TC-04, TC-05, TC-06, TC-15	100% Covered
REQ-CALL-003	Masked calling through Exotel without real phone leakage.	TC-07, TC-08	100% Covered
REQ-PAY-004	Payment webhooks verify signature and credit user minutes.	TC-09, TC-10	100% Covered
REQ-ADM-005	Admin updates case details, SLA tags and resolves complaints.	TC-11	100% Covered
REQ-SEC-006	Enforce parameterization, CSRF tokens, and script filtering.	TC-12, TC-13, TC-14	100% Covered